Artificial World — Documento completo

Un solo documento que integra todo: idea, ejecución, relato inversores, guía técnica y crónica fundacional. Para colgar en la web.

Fecha: 2026-03-08

Artificial World — Documento completo para web

> ***Un solo documento*** *que integra todo: idea, ejecución, relato inversores, guía técnica y crónica fundacional. Para colgar en la web.*

Fecha: 2026

Marca: Artificial World

Repositorio: artificial-word

PARTE 1 — La idea en 30 segundos

¿Qué es Artificial World?

Un mundo virtual donde los personajes piensan, recuerdan y se relacionan.

Como un acuario digital: pones criaturas, les das reglas, y ellas deciden qué hacer. Comen cuando tienen hambre, huyen del peligro, comparten con amigos y desconfían de quienes les han robado.

La diferencia: No usa ChatGPT ni nada parecido. Cada decisión es calculada, predecible y gratis. Corre en tu ordenador, sin internet.

En una frase

> ***No persigas la IA. Construye un mundo que la necesite.***

Tres palabras clave

| | |

|---|---|

| **Memoria** | Los personajes recuerdan lo que vieron y lo que pasó |

| **Relaciones** | Confianza, miedo, hostilidad que cambian con el tiempo |

| **Local** | Todo en tu PC. Cero coste por uso |

PARTE 2 — Resumen ejecutivo

Artificial World es una base para crear civilizaciones vivas con memoria, héroes, refugios y comunidades. La verdad estratégica vive en 2D; la encarnación 3D es capa futura.

Tesis: *Empieza con un refugio. Elige una semilla. Mira nacer tu civilización.*

Objetivo final: Motor creador de mundos compacto y reutilizable.

Qué existe hoy (verificado)

Componente	Estado	Evidencia
Motor Python 2D	Real	`principal.py`, 13 acciones, persistencia SQLite
Modo Sombra	Real	`gestor_modo_sombra.py`, tests
Web fullstack	Demo	Backend 3001, Frontend 5173
HeroRefuge	Parcial	7 semillas, mundos ligeros, companion IA
Panel Administrador	Real	`AdminPanel.jsx`, ruta `#admin`
DobackSoft	Demo	UI en hub, FireSimulator
IA local	Parcial	`aiCore.js`, Ollama, `/api/ai/*`
CI GitHub	Real	Tests, deploy Pages, deploy VPS
11 suites Python	Real	`run_tests_produccion.py`

Qué no existe

- 3D runtime
- Integración Python/JS
- DobackSoft comercial completo en este repo

PARTE 3 — Para tu abuelo (y para cualquiera no técnico)

Imagina un pueblo pequeño

En ese pueblo hay vecinos. Cada uno tiene su casa, su huerto y sus recuerdos.

- **Si ayudas a uno**, te guarda confianza. La próxima vez te tratará mejor.
- **Si le robas**, te tendrá miedo. Huirá cuando te vea.
- **Si no comes**, te debilitas. Si no descansas, te cansas.

Artificial World es ese pueblo, pero en pantalla. Los vecinos son personajes que el ordenador controla siguiendo reglas. No son aleatorios: cada acción tiene una razón (hambre, miedo, amistad).

¿Para qué sirve?

Para que los videojuegos y las simulaciones tengan personajes que **importan**. Que te reconozcan. Que cambien según lo que hagas.

¿Quién puede usarlo?

Cualquiera que quiera un mundo con vida. Estudiantes, creadores de juegos, empresas que simulan comportamientos.

PARTE 4 — Para inversores y jefes

El problema que resuelve

Los personajes de muchos juegos y simulaciones son **estáticos**. Dicen lo mismo, no recuerdan, no evolucionan. Los jugadores lo notan.

Las alternativas con IA generativa (ChatGPT, etc.) **cuestan dinero por uso**, tienen latencia y son impredecibles. No escalan para mundos con cientos de personajes.

La solución

| Artificial World | Competencia (LLM) |

|----------|----------|

| **Coste:** Cero por decisión | $ por token |

| **Latencia:** < 1 ms | 100–500 ms |

| **Predecible:** Sí | No |

| **Memoria persistente:** Sí | Limitada |

| **Escalable:** Miles de agentes | Coste crece con uso |

Mercado y oportunidad

- **Estudios indie** — NPCs creíbles sin presupuesto de API
- **Simulaciones** — Gestión, logística, formación
- **Educación** — Enseñar IA por utilidad
- **Investigación** — Motor abierto, modificable

Estado actual

- Motor funcional (Python + Web)
- 68+ tests pasando
- Ejecutable Windows (.exe) listo
- Documentación completa

PARTE 5 — Cómo ejecutar

Opción A — Motor Python (recomendado)

```
pip install -r requirements.txt
python principal.py
```

Resultado: Ventana pygame, entidades moviéndose, panel, Modo Sombra, guardar/cargar en `mundo_artificial.db`.

Opción B — Demo web

```
.\scripts\iniciar_fullstack.ps1
```

Resultado: Backend 3001, Frontend 5173, navegador abierto. Hub con HeroRefuge, DobackSoft, FireSimulator.

Opción C — Crónica fundacional (headless)

```
python cronica_fundacional.py --ticks 200
```

Resultado: `cronica_fundacional.json` y `cronica_fundacional.md`.

Opción D — Ejecutable Windows (sin instalar nada)

1. Descarga `MundoArtificial.exe`
2. Ejecútalo
3. Funciona igual que la versión completa

PARTE 6 — Stack técnico

Tecnologías

| Capa | Tecnología |

|------|----------|

| **Motor principal** | Python 3.11+, pygame |

| **Persistencia** | SQLite (mundo_artificial.db) |

| **Web** | Node.js + Express, React + Vite |

| **Tiempo real** | WebSocket |

Arquitectura del motor

```
nucleo.Simulacion → tick_runner
    ├── mundo/ (mapa, celdas, recursos)
    ├── entidades/ (EntidadSocial, EntidadGato)
    ├── agentes/ (MotorDecision, memoria, relaciones)
    ├── acciones/ (13: mover, comer, compartir, robar, huir, atacar...)
    └── sistemas/ (persistencia, watchdog, logs)
```

Cómo decide un agente

1. **Genera** acciones candidatas (mover, comer, ir al refugio...)
2. **Puntúa** cada una por utilidad (hambre, energía, rasgo, relaciones)
3. **Selecciona** la de mayor puntuación
4. **Ejecuta** y actualiza memoria/relaciones

Sin LLMs. Todo determinista y trazable.

PARTE 7 — Modelo conceptual (civilizaciones vivas)

| Entidad | Descripción |

|---------|----------|

| **World** | Contenedor: identidad, recursos, tick, crónica, semilla, refugio fundador |

| **CivilizationSeed** | Valores, tensiones, arquetipo, tono 2D/3D, héroe probable |

| **Refuge** | Unidad base: nombre, recursos, seguridad, moral, memoria local |

| **Hero** | Agente histórico: nombre, rol, arquetipo, lealtades |

| **Community** | Agrupación: cultura, tensiones, normas, cohesión, liderazgo |

| **MemoryEntry** | Registro: tipo, alcance, resumen, fecha |

| **HistoricalRecord** | Evento: tipo, título, resumen, significancia |

Flujo fundador: semilla → refugio → civilización naciente.

Regla 2D / 3D

- **2D** = Verdad sistémica (implementada)
- **3D** = Encarnación futura (roadmap)

PARTE 8 — Relato para inversores

Elevator pitch

Artificial World es un motor de mundos virtuales 2D persistentes con agentes que tienen memoria y relaciones. No usa IA generativa de pago: cada decisión es local, trazable y sin coste por uso. El producto ya existe: motor Python completo, demo web, ejecutable Windows, 68+ tests. La oportunidad: convertirse en el estándar de referencia para emuladores de mundos artificiales.

Líneas de monetización plausibles

1. **SDK / licencia** — Motor integrable para estudios de videojuegos (indie, AA)
2. **SaaS vertical** — DobackSoft como producto B2B: telemetría, estabilidad, formación
3. **Consultoría / laboratorio** — Simulación de equipos, dinámicas organizativas
4. **Educación** — Licencia para universidades, bootcamps, cursos de IA explicable
5. **Open core** — Motor base abierto; extensiones, soporte o hosting de pago

Manifiesto breve

Un mundo artificial no es solo un decorado. Es un espacio donde las cosas pasan, se recuerdan y tienen consecuencias. Un personaje que no recuerda no es un personaje: es un decorado animado.

Artificial World no pretende ser la simulación definitiva. Pretende ser una base donde el estado importa, donde la memoria importa y donde cada decisión puede dejar una historia verificable.

PARTE 9 — Tests y verificación

Python

```
$env:SDL_VIDEODRIVER="dummy"; $env:SDL_AUDIODRIVER="dummy"
python pruebas/run_tests_produccion.py
python pruebas/verificar_todo.py
```

Suites Python (11): estructural, core, crónica fundacional, modo sombra, combate, interacciones sociales, bug robar, watchdog fixes, watchdog integración, arranque limpio, integración producción.

Backend y frontend

```
cd backend; npm test
cd frontend; npm test
```

Backend: 39 tests (Vitest). **Frontend:** 6 tests (Vitest).

PARTE 10 — Crónica fundacional (ejemplo)

La crónica fundacional es un artefacto generado por `python cronica_fundacional.py`. Documenta el estado de una sesión reproducible: entidades, alertas, hitos y veredicto.

Ejemplo de estructura:

- **Fecha** — Timestamp de la sesión
- **Veredicto** — tension | prosperidad | supervivencia
- **Resumen** — Estado global en una frase
- **Metadata** — Semilla, fundador, refugio, ticks
- **Hitos** — Tick de inicio y cierre
- **Entidades finales** — Nombre, hambre, energía, posición
- **Alertas** — Eventos del watchdog (hambre, energía crítica, etc.)

Para generar tu propia crónica:

```
python cronica_fundacional.py --founder "Tu nombre" --refuge "Tu refugio" --ticks 200
```

PARTE 11 — Regla de foco y criterio de éxito

Regla de foco

Cada cambio debe responder:

¿Fortalece o distrae de "crear un refugio inicial y ver nacer una civilización"?

Si distrae → posponer.

Criterio de éxito

Una persona nueva entiende en menos de 5 minutos:

- qué es real
- qué es demo
- qué debe probar primero

Cierre

Artificial World en tres líneas

1. **Es** un motor de vida artificial donde los personajes piensan, recuerdan y se relacionan.
2. **No usa** ChatGPT ni APIs de pago. Todo corre local, gratis y predecible.
3. **Sirve** para juegos, simulaciones, educación e investigación.

Para cada persona

Si eres...	Tu siguiente paso
Abuelo / no técnico	Pide que te enseñen la demo en pantalla
Jefe / inversor	Revisa la parte 4 (valor y mercado)
Informático	Clona el repo y ejecuta `python principal.py`
Cualquiera	Comparte este documento con quien creas que le interese

Frase final

> ***Constrúyelo. Habítalo. Haz que crezca.***

Artificial World

Simulación de vida artificial 2D con agentes autónomos

artificial-word — El repositorio. Artificial World — La tesis. Motor creador de mundos — El destino.

Artificial World — Constrúyelo. Habítalo. Haz que crezca.